

SE2030

Software Engineering

Lecture 1: Introduction to Software Engineering, Ethics & SDLC Overview

Easy-to-understand • Complete lecture coverage • Exam-focused • Memory tips

HOW TO USE THIS NOTE: Read the normal explanation first, then revise only the EXAM IMPORTANT and MEMORY TIP boxes before the exam.

Source: Week1_Lecture1 - Student Version.pdf (55 pages)

1. Lecture Map & Learning Outcomes

- Define Software Engineering and explain how it differs from traditional programming.
- Describe the main SDLC phases.
- Explain the purpose and types of feasibility studies.
- Identify functional, non-functional, and constraint requirements.
- Recognize stakeholders and common requirement-gathering techniques.
- Explain ethics in software engineering and the role of IEEE/ACM professional codes.

Main topics in this lecture

Topic	What you must understand
Software Engineering	Meaning, importance, SE vs programming, software crisis
Feasibility Study	Purpose, output, five types
SDLC	Requirements, design, implementation, testing, deployment, maintenance
Requirements	Gathering techniques, requirement types, stakeholders, best practices
Ethics	Ethical values, IEEE/ACM, other professional codes

EXAM IMPORTANT: Be able to define the bold terms and give one simple example for each. Comparison questions and 'identify the type' questions are very likely from this lecture.

2. What is Software Engineering (SE)?

Software Engineering (SE) means applying engineering principles to build software that is reliable, efficient, and easy to maintain.

What SE focuses on

- Following structured processes.
- Ensuring quality at every step.
- Planning across the entire software lifecycle.

Why Software Engineering matters

- Saves time and money in the long run.
- Protects users and improves user satisfaction.
- Helps avoid major failures in critical systems.

Lecture examples: banking apps, hospital systems, and flight-control software.

MEMORY TIP: SE = Structured Engineering for the whole Software lifecycle. Think: Reliable + Efficient + Maintainable.

EXAM IMPORTANT: A short definition should include BOTH ideas: engineering principles + the goal of building reliable/maintainable software.

SE vs Programming

Aspect	Programming	Software Engineering
Focus	Code for specific tasks/features	Complete, reliable, maintainable systems
Scope	Narrow and task-oriented	Broad; whole software lifecycle
Activities	Coding, debugging	Analysis, design, architecture, coding, testing, deployment, maintenance
Teamwork	Often individual	Collaboration and team coordination
Process	May be informal/ad hoc	Structured, well-defined processes such as SDLC

Aspect	Programming	Software Engineering
End goal	Working code	Reliable, scalable, user-validated software
Real-world use	Scripts/prototypes	Large critical systems such as banking, healthcare, aviation

MEMORY TIP: Programming = one activity. Software Engineering = the complete process around that activity.

3. Software Crisis and the Need for SDLC

The Software Crisis (1960s-1990s)

Many software projects failed because they were:

- Over budget.
- Late / missed deadlines.
- Affected by crashes and serious disasters.

Failure	What happened	Why it failed
Therac-25	Patients received massive radiation overdoses	Poor testing and missing safety checks
Ariane 5 Rocket	Exploded about 40 seconds after launch	Reused code without proper adaptation
Mariner 1 (NASA)	Veered off course and self-destructed	A missing hyphen in code
Knight Capital	Lost \$440 million in 30 minutes	Wrong code update in stock-trading bot

EXAM IMPORTANT: The key lesson is not only the names of failures. Explain that lack of proper engineering structure, testing, adaptation, and change control can cause expensive or dangerous failures.

What is SDLC?

Software Development Life Cycle (SDLC) is a structured roadmap for building software from planning to maintenance.

- Helps meet user needs and expectations.
- Saves time and cost through planning.
- Reduces risks and delivery delays.

LECTURE ANALOGY: Building software without SDLC is like building a house without a blueprint. The customized-chair example also shows that everything starts by understanding the real need.

Overall flow used in the lecture

Feasibility -> Requirements -> Design -> Implementation -> Testing -> Deployment -> Maintenance

WATCH OUT: The lecture lists 6 key SDLC phases from Requirements to Maintenance, while the final summary shows Feasibility Study first in the overall flow. For this lecture, remember feasibility happens before actual development and leads into the SDLC work.

4. Feasibility Study

A feasibility study is done before actual software development. It answers the big question: 'Should we build this system?'

- Checks whether the project is technically possible.
- Checks whether it is financially affordable.
- Checks whether it is realistic.

Output: a Feasibility Report used by managers or stakeholders to approve or reject the project.

Five types of feasibility

Type	Main question	Lecture example
Technical	Can we build it with the technology we have?	Can current network/server setup support the new system?

Type	Main question	Lecture example
Economic	Is it worth the cost?	Will long-term benefits outweigh time and money spent?
Legal	Is the project legally safe?	Does it comply with GDPR/data-protection laws?
Operational	Will users actually use it?	Will staff accept and adapt to the system?
Schedule	Can we deliver it on time?	Can it go live before the new school year?

MEMORY TIP: Use TELOS: Technical, Economic, Legal, Operational, Schedule.

EXAM IMPORTANT: For a scenario question, do not only name the type. State the concern as a question, e.g., 'Do we have the hardware and skills?' = Technical feasibility.

Lecture activity: Apply all five feasibility types to a mobile app for rural farmers that provides weather updates, market prices, and farming advice.

5. Phase 1 - Requirements Gathering & Analysis

Main purpose: find out what the user/client wants the system to do and record it clearly in a Software Requirements Specification (SRS).

Who is involved?

- Clients.
- Users.
- Business analysts.

School management example: collect input from admin staff, teachers, and students.

Role of requirements gathering

- Happens right after the feasibility study.
- Provides the foundation for design, implementation, and testing.
- Acts like a contract between client and development team by stating what will and will not be built.

VISUAL LESSON FROM THE SWING CARTOON: Different people can understand the same request differently. Clear requirements prevent the final system from being different from what the customer actually needed.

Common requirement-gathering techniques

Technique	Simple meaning / use
Interviews	One-to-one or small-group conversations to understand needs and expectations
Questionnaires	Forms/surveys to collect feedback from a larger group
Observations	Watch users perform daily tasks to find problems and inefficiencies
Document analysis	Review reports, forms, manuals, workflows, and existing rules
Workshops / focus groups	Group discussions to agree priorities and clarify conflicts
Prototyping	Show mockups/sample screens and collect early feedback
Brainstorming	Generate ideas, features, and possible solutions with stakeholders

MEMORY TIP: Think: Ask, Watch, Read, Discuss, Show, Create. Interviews/questionnaires ASK; observation WATCHES; document analysis READS; workshops DISCUSS; prototyping SHOWS; brainstorming CREATES ideas.

Types of requirements

Type	Meaning	Lecture example
Functional	WHAT the system should do	The system should allow users to log in.
Non-functional	HOW the system should perform/behave	The system should load within 3 seconds.

Type	Meaning	Lecture example
Limitation / Constraint	Rule, restriction, or boundary	The system must run on Android only.
MEMORY TIP: WHAT - HOW - LIMIT = Functional - Non-functional - Constraint.		

Student Attendance Management System example

- Functional: teachers can mark daily attendance for each student.
- Non-functional: system is accessible on desktop and mobile.
- Constraint: system is deployed only within the university local network.

6. Stakeholders and Requirement Quality

Stakeholder: any person or group with an interest in the software system; they may use it, manage it, affect it, or be affected by it.

Types of stakeholders

Type	Meaning	Examples from lecture
End users	Directly use the system	Students using a school app; e-commerce customers
Internal users	Staff inside the organization who support/manage it	Admin staff, teachers, HR officers
External partners	Third parties the system interacts with	Payment gateways, suppliers, delivery services
Regulatory bodies	Authorities that set rules/standards	Education ministry, data-protection authorities
Project sponsor / client	Funds or owns the system	University board, government agency, company CEO

Best practices in requirement gathering

- Involve all key stakeholders early.
- Use clear, simple language and avoid unnecessary technical jargon.
- Validate requirements using reviews or walkthroughs.
- Document assumptions and limitations clearly.
- Maintain traceability from requirements through design, development, and testing.

Walkthrough: a peer-review meeting where the team goes through requirements, design, or code step by step to identify issues.

Common mistakes

- Guessing stakeholder needs without asking or confirming.
- Using too much technical jargon.
- Not documenting assumptions or limitations.
- Ignoring non-functional requirements such as speed, security, or usability.
- Failing to update requirements when the project changes.

EXAM IMPORTANT: A strong answer often pairs a best practice with the opposite mistake, e.g., 'validate requirements' versus 'guess stakeholder needs.'

Lecture activity - School Management System

You should be able to: identify 2 functional requirements, 2 non-functional requirements, 2 constraints, the stakeholders, and one suitable requirement-gathering technique.

7. Phase 2 - System Design

Design translates the SRS into a system blueprint. It breaks the required behavior into design components.

- UI Design - screen layouts and user flow.
- Database Schema - tables, relationships, and data structure.
- System Architecture - how front-end, back-end, and APIs work together.

Outputs: high-level design (overall structure) and low-level design (detailed logic/components).

Example: design the screens, database, and system structure for a student registration portal.

8. Phase 3 - Implementation (Coding)

- Developers write the actual code based on design documents.
- Follow coding standards and use suitable tools/technologies.
- Build the system module by module, e.g., login, registration, reports.
- Each module is tested individually using unit testing.

Login example: username/password fields, input validation, and error messages for invalid logins.

MEMORY TIP: Design = blueprint. Implementation = build the blueprint.

9. Phase 4 - Testing

Goal: find and fix bugs and confirm that software works as expected according to requirements.

Testing type	What it checks
Unit Testing	Each small function/module works correctly
Integration Testing	Different modules work correctly together
System Testing	The entire system behaves correctly
Acceptance Testing	The system meets the client's requirements

MEMORY TIP: U-I-S-A: Unit -> Integration -> System -> Acceptance. Think from smallest piece -> connected pieces -> whole system -> client approval.

Lecture example: Test a student registration form using valid details and invalid inputs such as missing fields or wrong formats, then check the response.

10. Phase 5 - Deployment

- Install the finished software in the real environment.
- May begin with a pilot release to a small group.
- Train users and prepare user manuals/help documents.
- Provide support through help desks, technical support, or FAQs.

Example: install the school management system on the school's network server so staff and teachers can use it.

11. Phase 6 - Maintenance

Maintenance starts after deployment. It keeps software functional, secure, and up to date through bug fixes, feature updates, and improvements.

- Changes may be needed because of new user needs.
- Technology may change.
- New security risks may appear.

Example: fix a login issue or add a new admin report feature.

Types of software maintenance

Type	Meaning	Lecture example
Corrective	Fix bugs/errors after deployment	Fix a crash during login
Adaptive	Change software for a new environment	Support a new Android/iOS version
Perfective	Improve performance or add features	Add an online-payment option

MEMORY TIP: CAP: Corrective = Correct problems; Adaptive = Adapt to environment; Perfective = Perfect/improve the system.

12. Ethics in Software Engineering

Ethics is the set of moral values used to decide what is right or wrong. In software engineering, ethics guides how professionals design, develop, test, and use software systems.

KEY IDEA: Being a software engineer is not only about writing code; it also means being responsible, fair, and professional.

Key ethical values in SE

Value	Meaning
Responsibility	Take ownership of your work and its impact
Transparency	Be honest about how the software works
Privacy	Protect users' personal information
Fairness	Avoid discrimination or bias

Lecture example: Even if you can access user data, you should not share it without permission. This is an ethical boundary, not only a technical issue.

MEMORY TIP: RTPF: Responsibility, Transparency, Privacy, Fairness.

Professional Codes of Ethics

Professional organizations provide formal codes to guide responsible behavior.

Organization / code	Lecture points
IEEE	Institute of Electrical and Electronics Engineers; global engineering/computing/electronics body; publishes standards such as IEEE 802.11; code noted as last updated 2020
ACM	Association for Computing Machinery; major computing-professional society; focus on computer science, education, software research; code last updated 2018
IEEE/ACM Joint Code	Created in 1999 specifically for software engineers; 'Software Engineering Code of Ethics and Professional Practice'
BCS Code of Conduct	UK; public interest, professional competence, accountability
ACS Code of Ethics	Australia; privacy, respect, social impact
IFIP Code of Ethics	International Federation for Information Processing; promotes ethical computing globally

Eight IEEE/ACM Software Engineering Code principles

1. Public
2. Client
3. Product
4. Judgment
5. Management
6. Profession
7. Colleagues
8. Self

MEMORY TIP: Memorize in four pairs: Public + Client | Product + Judgment | Management + Profession | Colleagues + Self.

EXAM IMPORTANT: Know that the IEEE/ACM joint code is specifically for software engineers and contains 8 core principles. Do not confuse it with the separate IEEE and ACM codes.

Lecture ethical dilemma

Scenario: management asks you to release software with known bugs just to meet a deadline.

Exam approach (study guidance, not a written answer from the slide): explain the risk honestly, prioritize public/client safety and product quality, document the concern, and avoid hiding known defects.

13. Exam-Focused Quick Revision

Question area	What you should be ready to write
Define SE	Engineering principles + reliable/efficient/maintainable software + lifecycle/process
SE vs programming	Programming is code-focused; SE covers the complete lifecycle and teamwork/process
Software crisis	Over budget, delays, crashes; examples show the need for disciplined SE/SDLC
Feasibility	Purpose, Feasibility Report, TELOS types + one example
SDLC order	Requirements -> Design -> Implementation -> Testing -> Deployment -> Maintenance; feasibility comes first in the lecture's overall flow
Requirements	Functional = WHAT; non-functional = HOW; constraint = LIMIT
Gathering techniques	Interviews, questionnaires, observation, document analysis, workshops, prototyping, brainstorming
Stakeholders	End, internal, external partners, regulators, sponsor/client
Testing	Unit, Integration, System, Acceptance
Maintenance	Corrective, Adaptive, Perfective
Ethics	Responsibility, transparency, privacy, fairness; professional codes
IEEE/ACM	1999 joint software-engineering code; 8 principles

Very short memory sheet

Topic	Memory trigger
SE	Reliable - Efficient - Maintainable
Feasibility	TELOS
Requirement types	WHAT - HOW - LIMIT
Testing	UISA
Maintenance	CAP
Ethical values	RTPF
IEEE/ACM 8 principles	Public, Client Product, Judgment Management, Profession Colleagues, Self

LAST-MINUTE EXAM TIP: When a question gives a scenario, first identify the keyword: cost = Economic, law = Legal, user acceptance = Operational, deadline = Schedule, technology/skills = Technical; system action = Functional, quality/performance = Non-functional, restriction = Constraint.

14. One-Minute Lecture Recap

- Software Engineering uses structured engineering practices to create reliable, maintainable software.
- The software crisis showed why disciplined processes and testing are necessary.
- Feasibility asks whether a project should be built before development begins.
- Requirements capture what users need; stakeholders must be involved and requirements must be clear and validated.
- Design creates the blueprint, implementation writes code, testing checks quality, deployment launches the system, and maintenance keeps it useful.
- Ethics requires responsible, transparent, private, and fair professional behavior supported by codes such as IEEE, ACM, and the IEEE/ACM joint code.

END OF LECTURE 1 NOTES